Multi-Layered Security Monitoring for Kubernetes and Containers

with Open Source Tools

BSc Thesis

September 2026

Vasileios Siampos

Supervisor Prof. Georgios Kambourakis

Committee Prof. S. Kokolakis; Assoc. Prof. G. Spathoulas

Dept. Information & Communication Systems Engineering

University University of the Aegean

A four-layer defence-in-depth architecture, validated against a deliberately vulnerable Kubernetes cluster.

https://github.com/SoloBary/Multi-Layered-Security-Monitoring-for-Kubernetes-and-Containers-with-Open-Source-Tools

What I will cover

1

The problem

Why Kubernetes defaults create risk

2

The argument

Where each tool goes blind

3

The architecture

Four layers, all open source

4

The attacks

Five scenarios mapped to MITRE ATT&CK

5

The measurements

Coverage, accuracy, operational cost

6

Limits & conclusions

What this does not claim

The problem is not exotic exploitation

Misconfiguration accounts for the majority of real-world Kubernetes compromise

What a default Kubernetes deployment provides to a workload — before you configure anything:

DEFAULT

Runs as root

UID 0 in the container is UID 0 on the host

DEFAULT

Token mounted

API credentials at a predictable path

DEFAULT

Flat network

Every pod reaches every other pod

DEFAULT

No limits

One workload can starve its neighbours

None of these are bugs.

They are documented defaults chosen to make the platform easy to adopt. Each one is also a step an attacker does not have to take on their own.
Tesla 2018 (dashboard without password) · Shopify 2018 (SSRF)

No single tool is sufficient

Each class of tool is effective within its scope — and blind outside it

Image scanner

Sees every CVE in the image

BLIND TO

Blind to everything after the container starts

Admission controller

Rejects workloads that violate policy

BLIND TO

Cannot tell a compromised pod from a healthy one

Service mesh

Enforces mTLS, exposes traffic flows

BLIND TO

A token read off local disk never touches the network

Runtime detection

Sees every syscall the workload makes

BLIND TO

Detects after the fact — it does not block

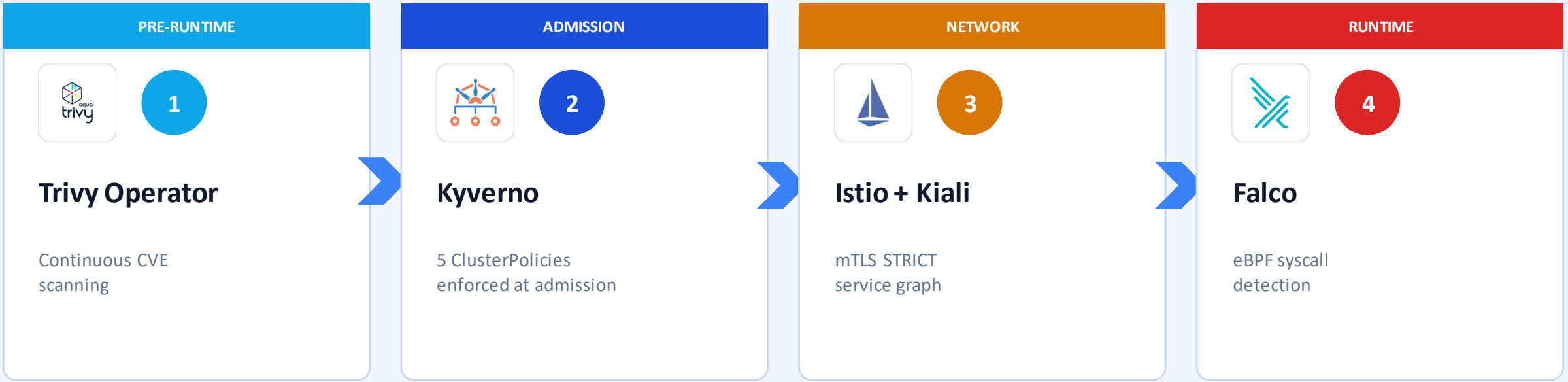
The blind spots are structural — they follow from where each tool sits in the lifecycle, not from poor engineering.

Research questions

RQ1	Can a complete defence-in-depth stack be built entirely from open source components?
RQ2	Which MITRE ATT&CK techniques does each layer catch, and where do coverage gaps remain?
RQ3	What is the operational overhead — particularly the runtime layer that processes every syscall?
RQ4	Is the coverage complementary, or are the layers just alerting on the same things?
RQ5	What tuning is required to make a default ruleset usable without creating blind spots?

The architecture

Four layers positioned across the workload lifecycle



Observability spine

Prometheus
metrics

Grafana
dashboards

Loki
logs

Counters tell you something happened. Log search tells you what.

TARGET

kube-goat

Deliberately vulnerable, documented misconfigurations

Monitoring, prevention, enforcement

Three different things the phrase 'security control' hides

MONITORING

Observes and records. Blocks nothing.

Trivy · Falco · Loki

IF IT FAILS

Failure: attack proceeds unobserved

PREVENTION

Refuses the operation before it takes effect.

Kyverno

IF IT FAILS

Failure: workload runs that should have been refused

ENFORCEMENT

Constrains what is possible during execution.

Istio mTLS STRICT

IF IT FAILS

Failure: execution of an operation that should be impossible

Adversary model: code execution already achieved inside a pod. From that foothold — escalate, harvest credentials, move laterally, persist.

The environment

Everything reproducible from the repository

```
$ kind create cluster --config kind-config.yaml

  name:  thesis-lab
  node:  kindest/node:v1.32.0
  kind:  v0.26.0

  extraMounts:  /proc  /sys  /
  # required for eBPF probe to reach
  # host kernel data structures
```

Deployed stack

Falco	0.43.0
Falcosidekick	2.32.0
Kyverno	v1.17.1
Trivy Operator	0.30.1
Istio + Kiali	1.24.0
kube-prometheus	v0.89.0
Loki Stack	v2.9.3

kube-goat — target

- Runs as root
- SA token mounted and exposed
- Images tagged :latest
- No CPU or memory limits

Why a vulnerable-by-design target

The admission layer needs genuine violations to reject. If I construct them myself, I am grading my own homework. kube-goat ships workloads that are misconfigured in specific, documented ways.

1 Pre-runtime — Trivy Operator

Continuous scanning as a controller, not a one-shot pipeline step

How it works

Runs as a Kubernetes controller. Scans every image in the cluster against CVE databases and publishes findings as VulnerabilityReport custom resources — queryable with kubectl like any other object.

Why the Operator, not a CLI scan

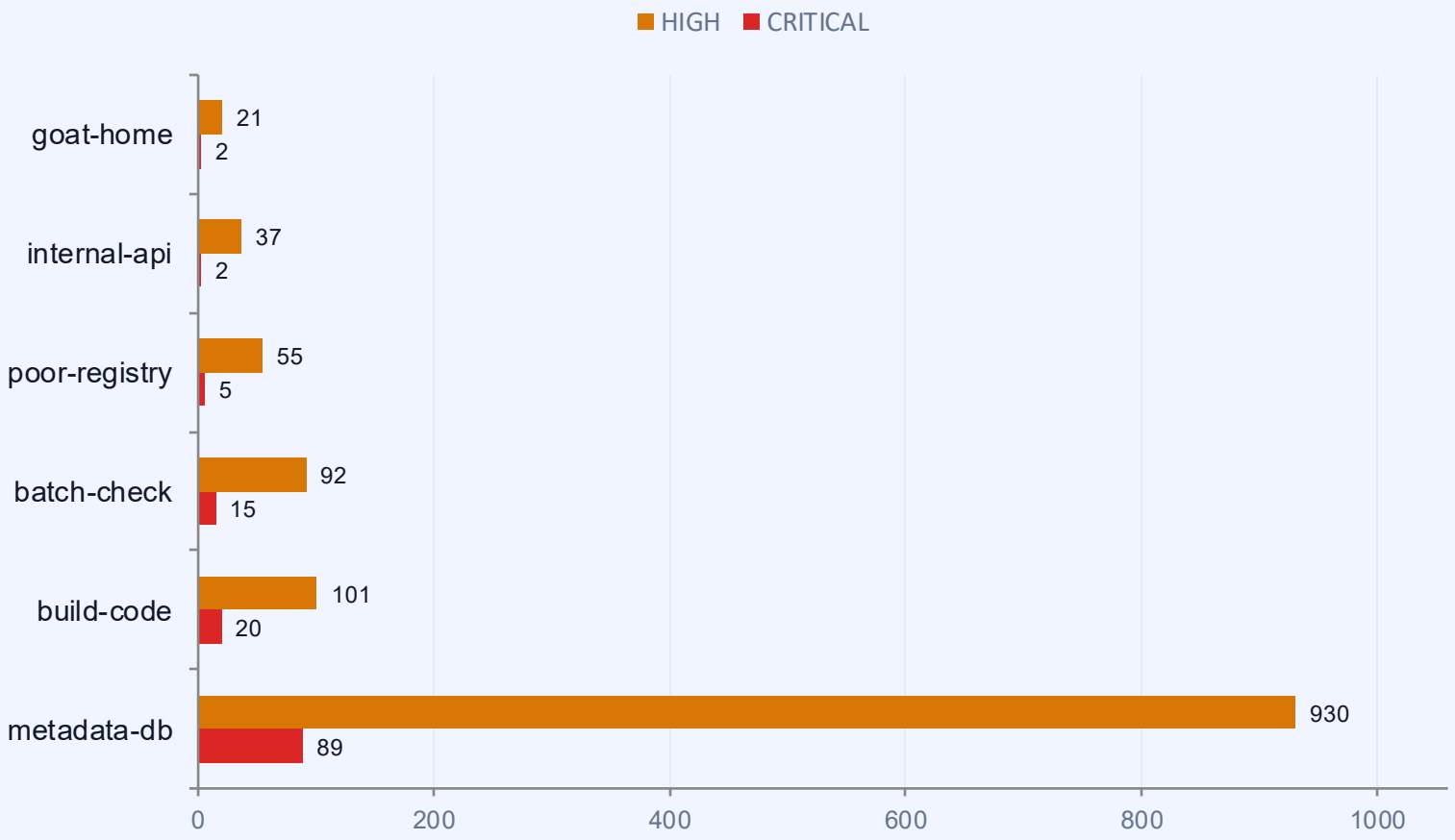
A pipeline scanner evaluates an image once, against the database as it stood that day. A workload deployed in March and still running in July has not been re-evaluated. The Operator re-scans on its own cycle — no human action required.

```
$ kubectl get vulnerabilityreports -A -o custom-columns=\
    NAMESPACE:.metadata.namespace,CRITICAL:.report.summary.criticalCount,HIGH:.report.summary.highCount

NAMESPACE    NAME                                  CRITICAL   HIGH
default      replicaset-metadata-db-...               89     930
default      replicaset-build-code-...                20     101
default      job-batch-check-job-batch-check          15      92
```

What the scanner found

Before a single attack command was executed



133

CRITICAL CVEs

across kube-goat workloads

1,245

HIGH CVEs

one image alone accounts for 930

2 Admission control — Kyverno

The last point at which a workload can be prevented from existing

`block-privileged`	privileged: true is refused
`require-non-root`	runAsNonRoot must be set
`block-latest-tag`	:latest is non-deterministic
`block-host-namespaces`	hostNetwork, hostPID, hostIPC
`require-resource-limits`	CPU and memory limits mandatory

Kyverno over OPA/Gatekeeper

These are structural assertions about pod specs. Expressing them in YAML that mirrors the object being validated is direct. Rego delivers the same enforcement with a second policy language to learn and maintain.

Namespace exceptions

System namespaces are excluded — falco, kube-system, monitoring, istio-system, trivy-system, kyverno. Without them, Kyverno blocks its own DaemonSets. The kube-goat namespaces are never excluded.

Enforcement against the real target

Redeploying a kube-goat workload with all policies in Enforce mode

```
$ kubectl apply -f /tmp/hunger-check.yaml -n big-monolith

Error from server: admission webhook "validate.kyverno.svc-fail"
denied the request:

resource Deployment/big-monolith/hunger-check-deployment was blocked
due to the following policies

require-non-root:
  validation error: Containers must not run as root.
    failed at path /spec/template/spec/containers/0/securityContext/
require-resource-limits:
  validation error: Resource limits are required.
    failed at path /spec/template/spec/containers/0/resources/limits/
```

34% violation rate

default namespace (kube-goat)

40% violation rate

secure-middleware (kube-goat)

0% violation rate

falco namespace — policies correctly excluded

3 Network — Istio and Kiali

Identity derived from workload, not from network location

STRICT over PERMISSIVE

PERMISSIVE accepts plaintext for gradual migration — it enforces nothing. STRICT rejects plaintext at the proxy. Every connection requires a valid certificate; identity comes from the service account, not from network location.

What this buys you

An attacker with execution in one pod cannot passively read traffic between two others, and cannot impersonate a service without also holding that service's certificate. Kiali renders the service graph, making cross-namespace traffic visible as an edge.

PeerAuthentication — all kube-goat namespaces

`default`	**STRICT**	100% mTLS
`big-monolith`	**STRICT**	100% mTLS
`secure-middleware`	**STRICT**	100% mTLS

4 Runtime — Falco on eBPF

Every privileged operation a workload performs crosses the syscall boundary

Kernel

eBPF probe at syscall entry/exit

Falco

Rules evaluated against event stream

Falcosidekick

Alerts to UI, Prometheus, Loki

Why modern_ebpf

- Kernel 6.8 has no prebuilt Falco driver
- kind nodes carry no kernel headers
- CO-RE resolves struct layouts against BTF at load time
- One object runs on any BTF kernel ≥ 5.8
- No init container, no compilation, no per-kernel artefact

Tuning: append, never disable

The default ruleset fires on kubelet mounts, Prometheus scrapes, node-exporter reads — all legitimate. Disabling those rules creates permanent blind spots. Instead, macros are extended with override: condition: append to whitelist specific system images and namespaces.

The kube-goat namespaces are never whitelisted.

Five attack scenarios

Executed against kubernetes-goat-home in the default namespace, mapped to MITRE ATT&CK

1	T1555	**Sensitive file access**	`cat /etc/shadow`	Read sensitive file untrusted
2	T1620	**Fileless execution**	`/dev/shm/evil`	Execution from /dev/shm
3	T1528	**Service account token**	`cat .../serviceaccount/token`	Read K8s Service Account Token *
4	T1105	**Command and control**	`curl -s http://example.com`	Detect curl or wget in container *
5	T1613	**K8s API lateral movement**	`curl -k https://kubernetes.default.svc`	Contact K8S API Server

** Custom rules — the default Falco ruleset has no detection for either. The absence of SA token detection is a finding in itself.*

Why scenario 2 matters

The technique that connects this lab to a real campaign

Fileless execution — T1620

/dev/shm is a memory-backed filesystem. A binary written there and executed leaves no artefact on disk — nothing to scan, no hash to match, no on-disk indicator. File integrity monitoring is defeated completely.

execve is still visible. The kernel must still be invoked to run a process

CanisterWorm — March 2026

A threat actor published malicious versions of Trivy and its GitHub Actions. A credential stealer paired with a self-propagating worm harvested npm tokens and republished infected packages autonomously. Spread across 141 packages.

Persistence mechanism: execution from /dev/shm.

Trace it through the stack

- **Trivy** flags the compromised package before deployment — if the malicious version is known
- **Falco** sees the /dev/shm execution regardless of how the binary arrived
- **Kiali** surfaces the outbound connection to C2 that has no reason to exist

MITRE ATT&CK coverage

Only techniques demonstrated with captured evidence are claimed

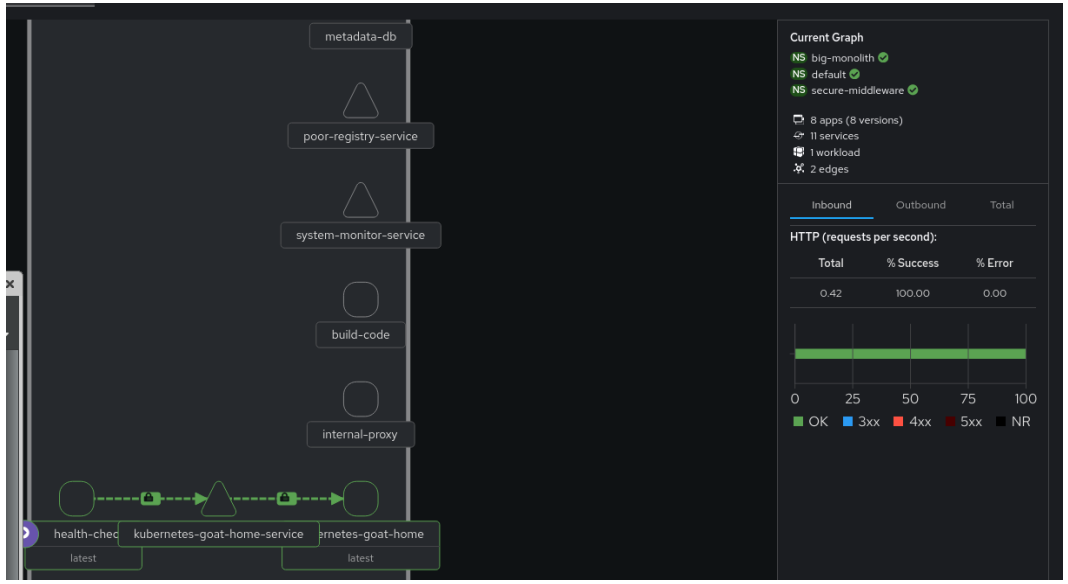
TECHNIQUE	DESCRIPTION	DETECTED BY	LAYER
T1195.002	Supply chain compromise	Trivy	Pre-runtime
T1078	Valid accounts — token exposure	Kyverno	Admission
T1555	Credentials from files	Falco	Runtime
T1620	Reflective code loading	Falco	Runtime
T1528	Steal application access token	Falco	Runtime
T1105	Ingress tool transfer	Falco	Runtime
T1613	Container and resource discovery	Falco + Kiali	Runtime + Network
—	Lateral movement — cross-namespace	Istio / Kiali	Network

Scope is strictly limited to techniques empirically validated in the lab environment..

The same cluster, seen from two layers

Captured live — both screenshots are in the repository

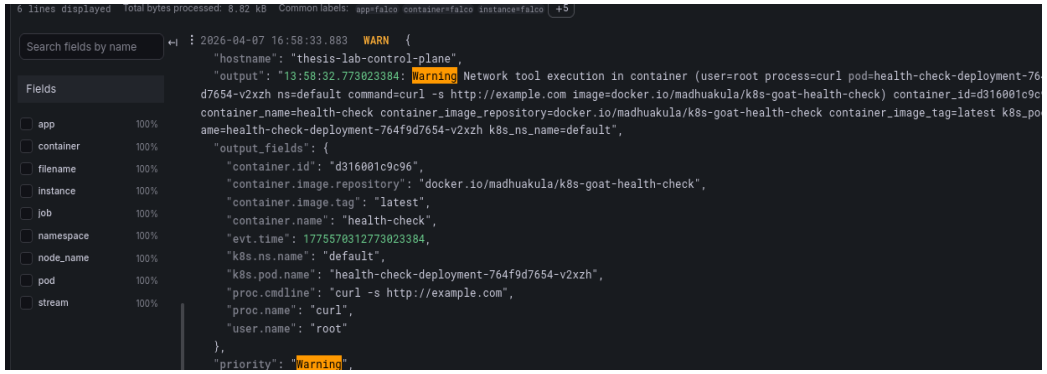
Istio / Kiali — mTLS on every edge



Three namespaces in the mesh, all green. Each padlock badge is mutual TLS.

Loki — the event behind the counter

```
{namespace="falco", container="falco"} |= "Warning"
```



Prometheus triggered an alert. Loki logs show curl example.com executed by health-check (root) in the default namespace, including the full command line.

This is why both are deployed.

What it costs to run

Falco processes every syscall on the node — measured live from Prometheus

CPU	MB RAM	syscalls/sec	event drops
5.5%	158	8,823	0
of one core, sustained	resident set size	sustained throughput	nothing missed

Zero drops is the number that matters

A dropped event is not a performance statistic — it is an attack that was never evaluated, and it leaves no trace. At 8,823 syscalls per second with zero buffer loss, nothing escaped the detection pipeline during the observation window.

Collected from Prometheus

```
falcosecurity_falco_cpu_usage_ratio * 100

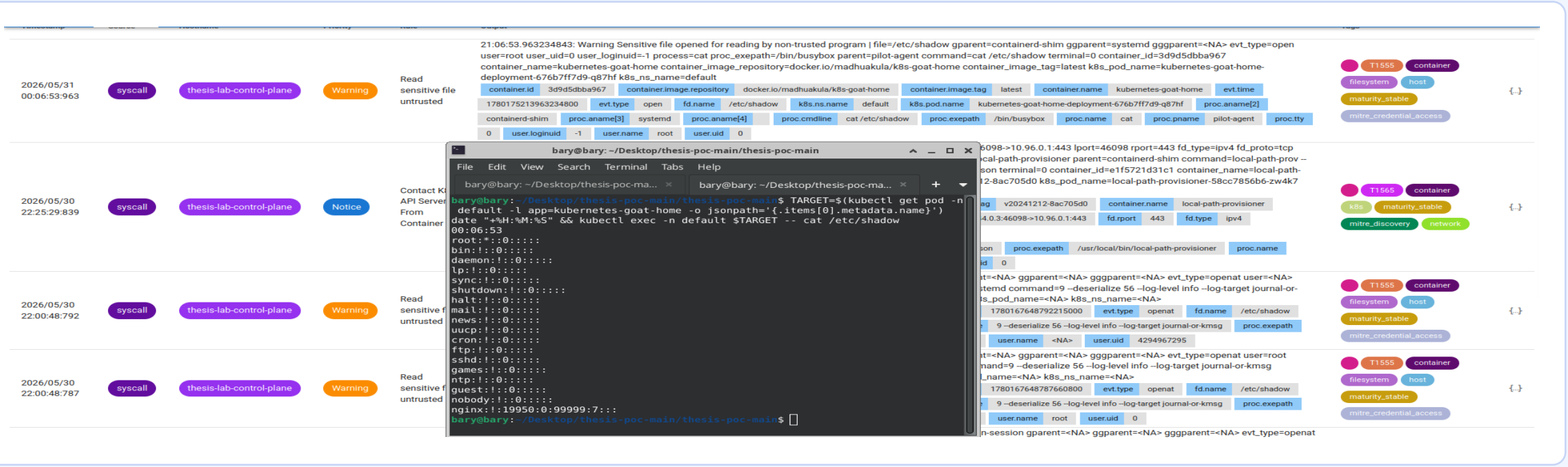
falcosecurity_falco_memory_rss_bytes

rate(falcosecurity_scap_n_evts_total[1m])

sum(falcosecurity_scap_n_drops_buffer_total)
```

Detection latency

From syscall to alert visible in the console



00:06:53

command issued

date "+%H:%M:%S" && kubectl exec … cat /etc/shadow

00:06:53.963

alert timestamp

Read sensitive file untrusted · T1555 · Warning

< 1 s

one measurement, not a distribution

What tuning actually achieved

Noise removed, coverage kept — not a count of alerts

BEFORE

kubelet, Prometheus, node-exporter
all firing alerts constantly

AFTER

Silent on legitimate activity —
attacks still fully detected

0

alerts from legitimate platform activity

after tuning — verified from Prometheus

No FP rate is presented — a meaningful before/after comparison needs a stable baseline of known benign activity, which this environment is too basic to provide. A percentage here would be a number without a method behind it.

Decisions I had to defend

Every choice in the repository has a recorded rationale

modern_ebpf

over legacy eBPF

CO-RE and BTF — no kernel headers, no compilation at startup, no per-kernel artefact to distribute

Kyverno

over OPA/Gatekeeper

Native YAML policies — no second policy language for structural assertions about pod specs

Macro append

over disabling noisy rules

A disabled rule is a permanent blind spot everywhere; an extended whitelist is narrowly scoped

mTLS STRICT

over PERMISSIVE

PERMISSIVE accepts plaintext for gradual migration — for a security demonstration it enforces nothing

What this work does not claim

Stated up front rather than discovered in questions

Single-node cluster

Multi-node scheduling attacks and node-to-node lateral movement are out of scope. All four layers behave identically regardless of node count, but distributed failure modes are untested.

Runtime detects, it does not block

Prevention happens at admission and at the network proxy only. Anything reaching runtime undetected is recorded, not stopped. Tetragon would have offered kernel-level enforcement.

No image provenance

No signature verification, no trusted digest baseline. A malicious image carrying no known CVEs passes the scanning layer without objection — the exact gap the CanisterWorm attack would exploit.

Attacks run without evasion

Commands were executed directly, in a manner a knowledgeable adversary would avoid. The detection results are an upper bound, not a claim about performance against an active evader.

Control plane hardening, etcd encryption and RBAC design are separate subjects and are not addressed here.

Conclusions

RQ1	Yes — a functioning four-layer stack assembled entirely from open source, no commercial component anywhere.
RQ2	Eight ATT&CK techniques covered with captured evidence, across all four layers. Gaps: provenance and correlation.
RQ3	5.5% CPU, 158 MB, zero event loss at 8,823 syscalls per second. Runtime cost is not the obstacle assumed.
RQ4	Complementary, not redundant. Scenario 2 would have been missed by three of the four layers. One caught it.
RQ5	Macro extension removed system noise entirely while preserving full coverage on the attack surface.

No single tool is sufficient — and the combination is buildable, affordable, and reproducible today.

Where this goes next

Image signing and provenance

Cosign and Kyverno verifyImages — closes the supply chain gap

Enforcement at runtime

Tetragon alongside Falco — kernel-level termination, not just alerting

Correlation

Alerts into a SIEM so slow-and-low activity becomes an incident

Multi-node evaluation

Scheduling attacks, node-to-node movement, behaviour under partition

Evasion-aware testing

Re-run scenarios with obfuscation to establish a realistic lower bound

Network policy

Default-deny between namespaces, closing the flat network model

Thank you

Questions welcome

Repository

https://github.com/SoloBary/Multi-Layered-Security-Monitoring-for-Kubernetes-and-Containers-with-Open-Source-Tools

Manifests · custom rules · PoC evidence · metrics

Vasileios Siampos

Dept. of Information & Communication Systems Engineering
University of the Aegean · September 2026